Design and Analysis of Algorithms (DAA)

Short theory, keywords and probable viva / exam questions for every practical

No.	Practical
1	Practical 1: Fibonacci Numbers (Recursive and Non-Recursive)
2	Practical 2: Huffman Encoding (Greedy Strategy)
3	Practical 3: 0/1 Knapsack (Dynamic Programming)
4	Practical 4: N-Queens (Backtracking, first Queen placed)
5	Practical 5: Quick Sort (Deterministic and Randomized)

Practical 1: Fibonacci Numbers (Recursive and Non-Recursive)

Topics in short

Fibonacci series: Each term is the sum of the previous two: 0, 1, 1, 2, 3, 5, 8, ... Recurrence: F(n) = F(n-1) + F(n-2) with base cases F(0) = 0, F(1) = 1.

Iterative (non-recursive) method: Uses a loop and two variables a, b. Time O(n), Space O(1). Storing all terms in an array costs O(n) space.

Recursive method: Function calls itself for n-1 and n-2. Recomputes the same sub-problems (overlapping sub-problems), so time grows exponentially, O(2^n) (tighter bound O(1.618^n)). Space O(n) because of the recursion stack depth.

Memoization / Dynamic Programming: Store each computed F(i) in a table so it is computed once. Top-down = memoization, bottom-up = tabulation. Time O(n).

Keywords

recurrence relation, base case, recursion stack, overlapping sub-problems, memoization, tabulation, time complexity, space complexity, O(2^n), golden ratio, tail recursion

Probable questions (with short answers)

Q1. What is the recurrence relation of Fibonacci?

Ans: F(n) = F(n-1) + F(n-2), F(0)=0, F(1)=1.

Q2. Why is recursive Fibonacci slow?

Ans: It solves the same sub-problems many times; number of calls grows exponentially.

Q3. Time and space complexity of recursive Fibonacci?

Ans: Time O(2^n), space O(n) (stack depth).

Q4. Time and space complexity of iterative Fibonacci?

Ans: Time O(n), space O(1) with two variables.

Q5. What is a base case? What happens without it?

Ans: Condition that stops recursion; without it the function recurses forever and gives a stack overflow (RecursionError in Python).

Q6. What is memoization?

Ans: Caching results of function calls so repeated calls return the stored value.

Q7. Difference between recursion and iteration?

Ans: Recursion uses function calls and the stack; iteration uses loops and no extra stack. Iteration is usually faster and uses less memory.

Q8. How many calls does fib_recursive(5) make?

Ans: 15 calls (calls(n) = 2*F(n+1) - 1).

Q9. Can Fibonacci be computed faster than O(n)?

Ans: Yes, matrix exponentiation or fast doubling in O(log n).

Q10. What is tail recursion?

Ans: Recursive call is the last operation; some languages optimize it into a loop.

Practical 2: Huffman Encoding (Greedy Strategy)

Topics in short

Greedy method: Makes the locally best choice at every step and never goes back. Works when the problem has the greedy-choice property and optimal substructure.

Huffman coding: Lossless compression. Frequent characters get short codes, rare ones get long codes. Codes are prefix codes: no code is a prefix of another, so decoding is unambiguous.

Building the tree: Put all characters in a min-heap by frequency. Repeatedly remove the two smallest, merge them into a new node whose frequency is their sum, push it back. Repeat n-1 times; the last node is the root.

Assigning codes: Traverse the tree: left edge = 0, right edge = 1. Path from root to a leaf = code.

Keywords

greedy choice, prefix code, variable-length code, min-heap, priority queue, lossless compression, binary tree, average code length, extract-min, optimal merge pattern

Probable questions (with short answers)

Q1. Why is Huffman coding called greedy?

Ans: At every step it merges the two lowest-frequency nodes, the best local choice.

Q2. What is a prefix code?

Ans: A code where no codeword is the prefix of another, e.g. 0, 10, 11.

Q3. Time complexity of Huffman coding?

Ans: O(n log n): n-1 merges, each heap operation O(log n).

Q4. Which data structure is used?

Ans: Min-heap / priority queue.

Q5. Is Huffman coding lossy or lossless?

Ans: Lossless: the original data is recovered exactly.

Q6. Bits needed for 'mississippi'?

Ans: 21 bits with Huffman instead of 88 bits with 8-bit ASCII.

Q7. Fixed-length vs variable-length code?

Ans: Fixed: every character same number of bits (ASCII). Variable: depends on frequency (Huffman), saves space.

Q8. Is the Huffman tree unique?

Ans: No, ties can be broken differently, but the total bits stay the same (optimal).

Q9. Where is Huffman coding used?

Ans: ZIP/DEFLATE, JPEG, MP3, PNG (as a stage of compression).

Q10. How is the average code length computed?

Ans: Sum of (frequency x code length) / total frequency.

Practical 3: 0/1 Knapsack (Dynamic Programming)

Topics in short

Problem: Given n items with weight and value and a bag of capacity W, choose items to maximize total value without exceeding W. Each item is taken fully (1) or not at all (0).

Dynamic programming: Breaks the problem into overlapping sub-problems and stores their answers in a table. Needs optimal substructure and overlapping sub-problems.

DP formula: dp[i][w] = max(dp[i-1][w], value_i + dp[i-1][w - weight_i]) if weight_i <= w, otherwise dp[i-1][w]. Answer is dp[n][W].

Traceback: If dp[i][w] != dp[i-1][w], item i was taken; subtract its weight and move up.

Keywords

dynamic programming, optimal substructure, overlapping sub-problems, tabulation, DP table, traceback, pseudo-polynomial, fractional knapsack, greedy vs DP, O(nW)

Probable questions (with short answers)

Q1. Why can't greedy solve 0/1 knapsack?

Ans: Choosing by best value/weight ratio can leave unused capacity; greedy is optimal only for the fractional knapsack.

Q2. Difference between 0/1 and fractional knapsack?

Ans: 0/1: whole items, solved by DP. Fractional: parts allowed, solved by greedy (sort by value/weight).

Q3. Time and space complexity?

Ans: O(nW) time and O(nW) space; space can be reduced to O(W).

Q4. Why is it called pseudo-polynomial?

Ans: O(nW) depends on the numeric value W, not on the input size in bits.

Q5. What are the two properties needed for DP?

Ans: Optimal substructure and overlapping sub-problems.

Q6. Memoization vs tabulation?

Ans: Memoization: top-down recursion with cache. Tabulation: bottom-up table filling.

Q7. What does dp[i][w] mean?

Ans: Best value using the first i items with capacity w.

Q8. Answer for weights {2,3,4,5}, values {3,4,5,6}, W = 5?

Ans: 7, by taking items 1 and 2.

Q9. Is 0/1 knapsack NP-complete?

Ans: Yes (decision version), though DP solves it in pseudo-polynomial time.

Q10. Other approaches?

Ans: Brute force O(2^n), branch and bound, backtracking.

Practical 4: N-Queens (Backtracking, first Queen placed)

Topics in short

Problem: Place N queens on an N x N board so that no two share a row, column or diagonal. Here the first queen is fixed by the user and the rest are placed automatically.

Backtracking: Build the solution step by step; when a step leads to a dead end, undo it (backtrack) and try the next option. It is DFS over the state-space tree with pruning.

Safety check: Same column: c1 == c2. Same diagonal: |r1 - r2| == |c1 - c2|. One queen per row is ensured by placing row by row.

Board representation: A 1-D list board[row] = column saves space instead of a full N x N matrix.

Keywords

backtracking, state-space tree, depth-first search, pruning, bounding function, constraint satisfaction, implicit/explicit constraints, diagonal check, O(N!)

Probable questions (with short answers)

Q1. What is backtracking?

Ans: A trial-and-error method that abandons a partial solution as soon as it breaks a constraint and tries the next choice.

Q2. How do you check diagonal attack?

Ans: Two queens attack diagonally if |row1 - row2| == |col1 - col2|.

Q3. Time complexity of N-Queens?

Ans: O(N!) in the worst case.

Q4. For which N is there no solution?

Ans: N = 2 and N = 3. N = 1 has one solution.

Q5. How many solutions for 4-Queens and 8-Queens?

Ans: 4-Queens: 2. 8-Queens: 92 (12 distinct).

Q6. Backtracking vs brute force?

Ans: Brute force tries every arrangement; backtracking prunes invalid branches early.

Q7. Backtracking vs branch and bound?

Ans: Backtracking uses DFS for feasibility; branch and bound uses bounds (often BFS/best-first) for optimization problems.

Q8. What is a state-space tree?

Ans: A tree of all partial solutions; each level = decision for one row.

Q9. Why might there be no solution when the first queen is fixed?

Ans: Some positions, e.g. a corner on a 4x4 board, cannot be part of any valid arrangement.

Q10. Other backtracking problems?

Ans: Sudoku, graph colouring, subset sum, Hamiltonian cycle.

Practical 5: Quick Sort (Deterministic and Randomized)

Topics in short

Quick sort: Divide and conquer: choose a pivot, partition so that smaller elements go left and larger go right, then recursively sort both sides. In-place, not stable.

Deterministic variant: Pivot is always a fixed position (last element). Sorted or reverse sorted input gives the worst case O(n^2).

Randomized variant: Pivot is a random element, so no particular input is always bad. Expected time O(n log n) for every input.

Partition schemes: Lomuto: pivot at the end, one index i. Hoare: two pointers from both ends, fewer swaps.

Keywords

divide and conquer, pivot, partition, Lomuto, Hoare, in-place, unstable, randomized algorithm, expected time, worst case O(n^2), average O(n log n), recursion depth

Probable questions (with short answers)

Q1. Best, average and worst case of quick sort?

Ans: Best and average O(n log n), worst O(n^2).

Q2. When does the worst case occur?

Ans: When the pivot is always the smallest or largest, e.g. sorted input with last-element pivot.

Q3. How does randomization help?

Ans: A random pivot makes balanced splits likely for any input; worst case becomes very unlikely.

Q4. Is quick sort stable?

Ans: No, equal elements may change their relative order.

Q5. Space complexity?

Ans: O(log n) average stack, O(n) worst case.

Q6. Quick sort vs merge sort?

Ans: Quick sort is in-place and faster in practice; merge sort is stable, always O(n log n), but needs O(n) extra space.

Q7. Recurrence for best and worst case?

Ans: Best: T(n) = 2T(n/2) + n. Worst: T(n) = T(n-1) + n.

Q8. What is a Las Vegas algorithm?

Ans: A randomized algorithm that always gives the correct answer; only its running time is random. Randomized quick sort is one.

Q9. Other ways to choose a good pivot?

Ans: Median-of-three, median of medians (guaranteed O(n log n)).

Q10. Why is quick sort fast in practice?

Ans: Good cache locality, small constant factors, in-place.